

Layer 02: Profiling — Findings and Decisions

Script: `sql/02_profiling.sql` | Status: Complete | Next layer: Core (`03_core.sql`)

1. Purpose

Profiling asks one question of the raw staging tables, over and over: *if this data were healthy, what would have to be true, and is it?* Nothing is changed here. The output of this layer is not clean data, it is evidence: a list of what is wrong, how much of it there is, and a rule for what to do about it. The rules below are what the core layer will implement.

2. How to read this document

Each finding has: what was measured, the number, what it means, and the decision. Where a finding confirms something already designed on purpose (Section 5 of the original methodology), it is marked **by design**. Where it is a genuine surprise, it is marked **new**.

3. Volume and shape

Check	Result
<code>staging_claims_raw</code>	1,014,550 rows, matches generation exactly
<code>staging_membership_raw</code>	1,214,303 rows, matches generation exactly
Distinct <code>claim_id</code> s	999,775 (fewer than row count)
Distinct members in claims	74,864
Distinct members in membership	49,975
Distinct providers	3,000
Claims date range	2022-01-01 to 2024-12-31 (full range, day-of-month bug fixed)

Decision: load confirmed clean. No further action. The gap between 74,864 and 49,975 members is investigated in Section 5 below, not treated as a loading error.

4. Grain and keys

Check	Result
Claim IDs appearing once	985,000
Claim IDs appearing twice	14,775 (never more than twice)
Of those pairs, disagree on <code>claim_status</code>	3,537
Disagree on <code>procedure_code</code>	1,272
Disagree on <code>member_id</code>	701
Disagree on <code>load_timestamp</code>	588
Disagree on <code>paid_amount</code>	141
Disagree on <code>patient_age</code>	81
Disagree on <code>region</code>	0
Member-months appearing once	1,190,259
Member-months appearing twice	12,022
Members with zero coverage gaps	24,487 (49%)
Members with 1 to 7 gap months	25,488 (51%)

Finding F1 — `claim_id` is not a valid key. *By design.* 1.5% of claims are duplicated.

Finding F2 — **duplicate claims are not identical copies.** *By design, but the specific disagreement pattern is new information.* A meaningful share of the 14,775 pairs disagree on a real field, most often `claim_status`. A plain `SELECT DISTINCT` would still leave both rows in whenever they differ, which would double-count dollars on those claims.

Decision D1: the true unique key for a claim going forward is a new surrogate key (`claim_sk`), not `claim_id`. `claim_id` is retained as a business field that can repeat.

Decision D2: deduplication uses a survivorship rule, not a blind `DISTINCT`:

1. Standardize values first (casing, spelling) so cosmetic differences stop counting as real disagreement.

2. Score each copy: does it have a known `member_id`, a non-blank `procedure_code`, a plausible `patient_age`, `paid_amount` not exceeding `billed_amount`, a well-formed `load_timestamp`.
3. Keep the higher-scoring copy. Tie-break on the lower `row_id` (earlier load wins).

Decision D3: `(member_id, month_year)` is deduplicated the same way in membership.

Decision D4 — coverage gaps, deferred. 51% of members have at least one missing month, but the maximum is 7 months out of 36, and the shape (many members with 1-2 gaps, few with 6-7) looks like scattered single-month drops rather than long real lapses. No fill rule is finalized yet: this will be revisited when the marts layer builds the exposure table, since the decision affects PMPM directly and deserves its own evidence (gap run-length, not just gap count).

5. Referential integrity

Check	Result
Orphan claim rows (member not in membership at all)	25,906 (2.55%)
Distinct orphan member IDs	25,387
Members with zero claims in 3 years	498 of 49,975 (1.0%)
Claims outside the member's enrolled month	34,442 (3.39%)

Finding F3 — orphan claims. *By design.* Matches the intentional ~2.5% injection rate from the methodology.

Finding F4 — members with no claims. *Expected, not a defect.* Not everyone files a claim in three years.

Finding F5 — claims outside enrollment window. *Originally a generator bug (32%), now fixed and reduced to 3.39%.* The residual matches the intentional coverage-gap injection rate and is treated as expected, documented in `Data_Generation_Addendum_1.md`.

Decision D5: claims whose `member_id` does not exist in membership at all are flagged `unmatched_member = 1` and excluded from any calculation that requires a real member (PMPM, member-level features). They are not deleted, since the claim dollars are still real and belong in aggregate totals that do not require member identity.

Decision D6: claims outside the member's enrolled month are flagged `outside_enrollment = 1` rather than excluded outright, pending the gap-fill decision in D4. If a 1-month gap gets filled as a likely load miss, some of these claims will reconcile automatically.

6. Completeness

Field	Blank count	% of rows	Expected?
<code>paid_date</code>	121,778	12.0%	Yes — equals denied (71,145) + pending (50,633) <code>(*)</code> exactly
<code>diagnosis_code_2</code>	709,899	70.0%	Yes — secondary diagnosis is optional by design
<code>procedure_code</code>	45,654	4.5%	Yes — matches the injected missingness rate
<code>revenue_code</code>	908,905	89.6%	Yes — equals every non-Inpatient row exactly
<code>length_of_stay</code>	891,204	87.8%	Yes — equals everyone except Inpatient and the emergency-observation subset
<code>paid_amount</code>	49,581	4.9%	Yes — equals pending claims, minus 777 that hold a value in error <code>(**)</code>
<code>denial_reason_code</code>	943,405	93.0%	Yes — populated only for denied claims
<code>original_claim_id</code>	999,556	98.5%	Yes — populated only for adjustment records
<code>load_timestamp</code>	5,179	0.5%	Partially — see Finding F8

`(*)` $71,145 + 50,633 = 121,778$, exact. `(**)` Denied should be \$0 exactly; 70,786 of 71,145 are, but 359 carry a nonzero value. Pending should be blank; 49,581 of 50,633 are, but 777 (Finding F9) carry a value or \$0.

Decision D7: blanks that are "not applicable to this claim type" (revenue code, length of stay, diagnosis 2) are left as `NULL` in the core layer, not imputed. A blank here is information (this field does not apply), not missing data.

Decision D8: a blank `procedure_code` is a genuine gap, not a "does not apply" case (every claim type shows the same ~4.5% missing rate). It is kept as `NULL` in core and flagged `procedure_code_missing = 1` as a feature, so a model can use "we don't know the procedure" as information rather than losing the row.

7. Validity

Check	Result
Non-numeric age	0
Negative age	1,553
Age over 120	1,490
Plausible age range	0.0 to 93.0
<code>paid_amount</code> > <code>billed_amount</code>	5,072
<code>paid_amount</code> > <code>allowed_amount</code>	8,385
<code>allowed_amount</code> > <code>billed_amount</code>	0
Dates: bad format, or paid before submitted, or submitted before service	0 in every case
<code>load_timestamp</code> well-formed	979,563 (96.6%)
<code>load_timestamp</code> malformed	34,987 (3.4%)
Records "loaded" before their claim was paid	40,196
Chronic/adjustment flags outside {0,1}	0

Finding F6 — negative and over-120 ages. *By design.* $1,553 + 1,490 = 3,043$, close to the intentional 0.3% injection rate stated in the methodology (the original target was smaller; actual landed a little higher, which is normal for a random injection).

Finding F7 — `paid_amount` exceeding billed or allowed. *By design,* matching the ~0.5% processing-error injection, though it also touches a small number of denied/pending claims that should have been \$0 or blank (see Finding F9).

Finding F8 — `load_timestamp` has two distinct formats, not one. *New.* Of the 34,987 "malformed" rows, only 15,111 are the genuinely broken values designed in (blank, `00/00/0000`, `2025-13-45`, `N/A`). The remaining ~19,876 are valid **dates without a time component** (e.g. `2025-02-23`), overwhelmingly on adjustment records. This is not random corruption, it is a second, real format that a different part of the generator produced.

Finding F9 — 40,196 rows where the load timestamp predates the paid date. *Known generator limitation, already documented in the addendum.* All load timestamps cluster in

Jan-Mar 2025 regardless of the claim's actual payment date, so a claim paid late in 2024 with a long processing lag can show as "loaded" before it was "paid." Not fixable without redesigning the load-timestamp scheme; not worth doing, since `load_timestamp` is only used for ordering duplicate versions of the same claim, not for lifecycle logic.

Decision D9: age is cast to a nullable numeric type. Values outside a plausible clinical range (below 0 or above 110) are set to `NULL` and flagged `age_invalid = 1`, not deleted or guessed at.

Decision D10: `paid_amount` is capped at `allowed_amount` where it exceeds it, and the original value is preserved in an audit column (`paid_amount_raw`) so the correction is traceable, not silently overwritten.

Decision D11: `load_timestamp` parsing in the core layer must accept both `YYYY-MM-DD HH:MM:SS` and `YYYY-MM-DD` formats. Rows matching neither are the true malformed set and are set to `NULL`.

8. Consistency

Claim status — every raw spelling accounted for, none left unrecognized:

Raw value	Rows	Maps to
Paid, paid, PD	$774,059 + 59,201 + 59,512 = 892,772$	<code>Paid</code>
Denied, DENIED, denied	$61,652 + 4,766 + 4,727 = 71,145$	<code>Denied</code>
Pending, PENDING, pending	$43,843 + 3,380 + 3,410 = 50,633$	<code>Pending</code>

Region — every raw spelling maps cleanly to one of four regions (South, Midwest, Northeast, West), confirmed across both files. No unexpected fifth region or ambiguous value appeared anywhere.

Finding F10 — `place_of_service` is fully redundant with `claim_type`. Every one of the five claim types maps to exactly one `place_of_service` value, with no exceptions across 1,014,550 rows (e.g. every single Emergency claim shows `Emergency Room`). The column carries no information beyond what `claim_type` already provides.

Finding F11 — status mostly agrees with the payment fields, with a small, explainable exception. Paid claims always have a paid date and zero blank amounts. Denied claims are \$0 in 70,786 of 71,145 cases; pending claims are blank in 49,581 of 50,633 cases. The mismatch in both (359 and 1,052 respectively) traces to the injected "paid > billed" error, which overwrites a value without checking claim status first — a realistic kind of bug, not one to silently patch away.

Decision D12: build a single standardization mapping (a `CASE` expression or lookup table) covering all nine raw `claim_status` spellings, all region variants, and apply it once in the core layer.

Decision D13: drop `place_of_service` from the core schema, or keep it only as a reference column with a one-line note that it duplicates `claim_type`. Not used as a feature.

Decision D14: the small claim-status/payment mismatches (Finding F11) are left as-is, not corrected — they are a genuine, realistic artifact and are more valuable documented than silently fixed.

9. Adjustments and duplicates

Check	Result
Original claim rows (<code>C</code> prefix)	999,556
Adjustment rows (<code>A</code> prefix)	14,994, all with <code>adjustment_flag = 1</code>
Adjustment rows naming an original	14,994 of 14,994 (100%)
Distinct original claims adjusted	14,775
Every named original found in the file	14,994 of 14,994 (100%)

Finding F12. Adjustment records have full referential integrity: every single one correctly points to a real, present original claim. The count (14,994) is slightly higher than the base design (14,775) because some adjustment rows themselves were caught by the later duplicate-injection step, which drew from the whole file, adjustments included.

Decision D15: an adjustment record replaces its original claim's financial values in the core layer (its `paid_amount` supersedes the original's), keeping the original row retrievable for audit via `original_claim_id`, not deleted.

10. Dimension candidates

Check	Result
Providers with more than one type, specialty, network status, or NPI	0 of 3,000, every case
Members (seen via claims) with varying sex, chronic flags, or region	0 of 74,864, every case
Members (seen via membership) with varying plan, birth date, sex, or region	0 of 49,975, every case

Finding F13 — this is the strongest result in the entire profiling exercise. Every provider and every member is perfectly stable on every attribute tested, with zero exceptions across the full dataset.

Decision D16 (the ER decision): `dim_provider` (type, specialty, network status, NPI) and `dim_member` (sex, chronic flags, region, plan, birth date) are confirmed as valid dimension tables. These attributes are stored once per entity and joined by key, not repeated on every claim row. The star schema drafted before profiling is confirmed by evidence, not just hypothesis.

11. Business shape

Claim type	% of claims	% of paid dollars
Inpatient	10.4%	68.1%
Emergency	11.7%	14.5%
Outpatient	37.4%	14.5%
Preventive	15.9%	1.5%
Pharmacy	24.6%	1.4%

Metric	Value
Mean paid amount	\$3,419
Median paid amount	\$660
99th percentile	\$43,930
Maximum paid claim	\$3,123,531
Top 1% of claims, share of total dollars	22.8%
Top 10% of claims, share of total dollars	71.4%

Claim type	Avg. days to submit	Avg. days to paid
Pharmacy	3.5	31.0
Preventive	7.5	35.0
Outpatient	11.0	38.5
Emergency	15.0	42.5
Inpatient	25.0	52.5

Monthly volume and dollars rise steadily from 2022 to 2024, with a visible seasonal bump each November-January.

Finding F14. Every design target from the original methodology is confirmed in the actual, loaded data: cost concentration, right-skew, and claim-type-dependent lag all match. This is the evidence base for the log-transform decision, the naive-baseline comparison, and the completion-factor/lag-triangle work still to come.

Decision D17: no changes needed. This section validates that the modeling work ahead (log transform, GLM, backtest, IBNR-lite) is being built on data that actually behaves the way the project's design intended.

12. Summary: every decision carried into the core layer

#	Decision
D1	Surrogate key <code>(claim_sk)</code> ; <code>(claim_id)</code> is a repeatable business field
D2	Dedup claims via standardize-then-score-then-keep-best, not <code>(DISTINCT)</code>
D3	Same dedup approach for <code>(member_id, month_year)</code>
D4	Coverage-gap fill rule deferred to the marts layer, pending gap run-length
D5	Orphan claims flagged, not deleted; excluded from member-dependent calculations
D6	Out-of-enrollment claims flagged, pending D4
D7	"Not applicable" blanks stay <code>(NULL)</code> , not imputed
D8	Missing <code>(procedure_code)</code> stays <code>(NULL)</code> plus a <code>(procedure_code_missing)</code> flag
D9	Implausible age set to <code>(NULL)</code> plus an <code>(age_invalid)</code> flag
D10	<code>(paid_amount)</code> capped at <code>(allowed_amount)</code> ; original preserved in an audit column
D11	Parse both <code>(load_timestamp)</code> formats; only true non-matches become <code>(NULL)</code>
D12	One standardization mapping for <code>(claim_status)</code> and <code>(region)</code>
D13	Drop or de-emphasize <code>(place_of_service)</code> (redundant with <code>(claim_type)</code>)
D14	Leave small status/payment mismatches as documented, not corrected
D15	Adjustment records supersede their original's financial values; original stays for audit
D16	<code>(dim_provider)</code> and <code>(dim_member)</code> confirmed by evidence
D17	No change; business shape validated

13. Interview one-liners

- **30 sec:** "Profiling found that duplicates weren't exact copies, so I built a standardize-then-score survivorship rule instead of a blind DISTINCT. It also confirmed, with zero exceptions across 3,000 providers and 75,000 members, that my dimension design was correct before I built it."

- **On the generator bug:** "I found a third of claims fell outside their member's enrollment window during profiling, traced it to a real logic gap in how I built the data, fixed it, and verified the fix brought it down to the expected 3.5% baseline."
- **On completeness:** "Every blank I found had a specific, checkable explanation. I didn't impute anything without first confirming whether a blank meant 'missing' or 'does not apply.'"